Multiplicacao de Matrizes em Blocos com MPI e OpenBLAS

Resumo

Foram compiladas e testadas as tres versoes do projeto no Lovelace. O teste de corretude com `N=128`, bloco `32` e dois processos terminou com `validation=ok` para `mpi_mm` e `mpi_blas_mm`. A campanha forte produziu 105 medicoes validas: 35 configuracoes, cada uma com tres repeticoes. Tambem foram executadas nove medicoes de escalabilidade fraca.

Os resultados sao reais e estao em `results/raw/` e `results/processed/`.

Ambiente experimental

• Host de login: `lovelace.ada.cenapad.unicamp.br`.

• Host PBS multinode: `adano05` e `adano27`.

• CPU reportada: AMD EPYC 7282 16-Core Processor.

• CPUs visiveis na sessao SSH: 1; a campanha PBS usou 128 slots por no.

• Memoria reportada: 62 GiB.

• Kernel: `4.18.0-553.150.1.el8_10.x86_64`.

• Compilador: GCC 9.4.0, modulo `gcc/9.4.0`.

• MPI: OpenMPI 4.1.1, modulo `openmpi/4.1.1-gcc-9.4.0`.

• CMake: 4.3.1, modulo `cmake/4.3.1`.

• OpenBLAS: modulo `openblas/0.3.21-gcc-9.4.0`; o executavel foi confirmado

com `ldd` usando `libopenblas.so.0`.

• Python: 3.6.8.

• Matplotlib: 3.3.4.

• Launcher usado: `/opt/pub/openmpi/4.1.1/gcc/9.4.0/bin/mpiexec`.

• `OPENBLAS_NUM_THREADS=1` foi definido nos testes MPI+OpenBLAS.

Os testes preliminares foram feitos diretamente no host SSH e sao identificados como tal nos logs. A campanha final multinode foi executada pelo PBS no job `1032047.ada`; seus recursos e nos estao documentados separadamente.

Correcoes realizadas

O CMake original nao propagava o diretorio `include/` para os executaveis MPI. As propriedades de include das bibliotecas `mpi_matmul_core` e `mpi_blas_core` foram alteradas para `PUBLIC`.

Tambem foi corrigido um erro de execucao em `src/mpi_matmul.cpp`: o buffer `gathered` so era alocado quando `--validate` estava ativo, embora `MPI_Gatherv` fosse sempre chamado. Isso causava `SIGSEGV` em qualquer benchmark MPI sem validacao.

O arquivo de instrucoes foi mantido como `instrucoe.txt`, que e o nome presente no repositorio recebido. A diferenca em relacao ao nome `instrucoes.txt` mencionado informalmente foi registrada para evitar ambiguidade.

Corretude

Comandos executados:

./build/serial_mm --n 128 --block 32 --validate
mpiexec -np 2 ./build/mpi_mm --n 128 --block 32 --validate
OPENBLAS_NUM_THREADS=1 mpiexec -np 2 ./build/mpi_blas_mm --n 128 --block 32 --validate

Resultados:

Versao	Max abs	Max relativo	Validacao (s)	Resultado
serial	2.86102295e-06	0.00520082	0.0026518	ok
MPI	2.86102295e-06	0.00520082	0.0026212	ok
MPI + OpenBLAS	6.67572021e-06	0.00440960	0.0027135	ok

O tempo de validacao foi medido depois da computacao e nao foi incluido em `compute_seconds` ou `total_seconds`.

A referencia usada agora e uma multiplicacao serial independente com os loops `i-j-k`, diferente da implementacao tiled usada para computar o resultado.

Campanha de desempenho

A campanha forte usou um host, `N=2000, 4000, 6000`, blocos `64, 128, 256` quando aplicavel, processos MPI `1, 2, 4` e tres repeticoes. Para `N=4000` e `N=6000`, foi usado bloco `128` para limitar o custo da campanha. Os benchmarks foram executados sem `--validate`, pois a validacao serial de uma matriz grande nao deve ser parte da campanha de desempenho.

Arquivos gerados:

- `results/processed/measurements.csv`: 105 linhas de medicao.
- `results/processed/statistics.csv`: 35 grupos estatisticos.
- `results/processed/speedup_efficiency.csv`: speedup relativo a serial.
- `results/plots/compute_time_n2000.png`.
- `results/plots/compute_time_n4000.png`.
- `results/plots/compute_time_n6000.png`.

Exemplos de medias de `compute_seconds` para bloco `128`:

N	Serial	MPI+BLAS p=1	MPI+BLAS p=2	MPI+BLAS p=4
2000	0.882474	0.235511	0.117759	0.059574
4000	6.810632	1.893012	0.945933	0.472410
6000	23.145013	7.264287	3.621751	1.609950

Esses valores sao medias das tres repeticoes e devem ser analisados junto com `statistics.csv`, que contem desvio-padrao, minimo e maximo.

Os blocos `64`, `128` e `256` foram comparados diretamente para `N=2000`. Blocos menores aumentam a quantidade de tiles e podem elevar a sobrecarga de loops e chamadas BLAS; blocos maiores reduzem essa sobrecarga, mas aumentam a pressao sobre cache. As medidas detalhadas e a variabilidade estao no CSV, permitindo comparar cada bloco sem misturar configuracoes.

As medias de `compute_seconds` da versao MPI+OpenBLAS foram:

Bloco	p=1	p=2	p=4
64	0.316600	0.159129	0.077233
128	0.235511	0.117759	0.059574
256	0.195526	0.099847	0.048887

Nesse ambiente, o bloco 256 foi o mais rapido nos tres niveis testados. Isso e uma observacao experimental para N=2000 e nao deve ser generalizada para matrizes ou arquiteturas diferentes sem nova medicao.

Escalabilidade forte

Foi mantido N fixo e aumentado o numero de processos entre 1 e 4. A referencia de speedup entre versoes foi a media serial da mesma matriz e do mesmo bloco. O speedup paralelo foi calculado como T(1)/T(p) e a eficiencia como speedup/p. Os valores completos estao em results/processed/speedup_efficiency.csv.

Os resultados preliminares de 1 a 4 processos no host SSH medem principalmente oversubscription. A campanha PBS multinode, descrita abaixo, e a referencia para o impacto de nucleos e nos.

Escalabilidade fraca

Foi usada a regra aproximada N proporcional a p^(1/3), mantendo bloco 128 e a versao MPI+OpenBLAS:

- p=1, N=2000;
- p=2, N=2520;
- p=4, N=3175.

Cada configuracao teve tres repeticoes. Os logs estao no diretorio results/raw/weak-* e os dados processados em results/processed/weak_measurements.csv.

Reprodutibilidade

Evidencias preservadas:

- results/environment/inventory/;
- results/environment/final-inventory/;
- results/environment/bash-syntax.txt;
- results/environment/build-final.txt;
- results/environment/build-after-mpi-fix.txt;
- results/environment/mpi_blas_ldd.txt;
- results/raw/correctness/;
- results/raw/final-*;
- results/raw/weak-*;
- results/raw/1032047.ada-20261019-163130-multinode/;
- results/processed/;
- results/plots/.
- scripts/process_results.py;
- scripts/generate_report_pdf.py.

O processamento reproduzivel e feito com:

python3 scripts/process_results.py --raw-dir results/raw/benchmark-final
python3 scripts/process_results.py --raw-dir results/raw/1032047.ada-20261019-163130-multinode --prefix multinode
python3 scripts/process_results.py --raw-dir results/raw/weak-20261019-145859 --prefix weak
python3 scripts/plot_results.py
python3 scripts/generate_report_pdf.py

Todos os scripts Bash e PBS passaram por bash -n. O arquivo report/report.pdf foi gerado com ReportLab e inclui os graficos produzidos.

Campanha PBS Multino

O script pbs/run_multinode.pbs foi executado pelo job 1032047.ada na fila paralela, com dois nos e 128 CPUs por no. O arquivo environment.txt registra os nos efetivamente alocados: adano05 e adano27, com 128 slots cada. Foram executadas 51 medicoes: tres repeticoes para serial e tres repeticoes para cada uma das versoes MPI nos niveis 1, 2, 4, 8, 16, 32, 48 e 96 processos.

Os resultados foram processados em:

- results/processed/multinode_measurements.csv;
- results/processed/multinode_statistics.csv.
- results/processed/multinode_speedup_efficiency.csv.

Para N=2000 e bloco 128, as medias de compute_seconds foram:

Processos	MPI	MPI + OpenBLAS
1	0.972636	0.234000
2	0.490366	0.117708
4	0.244848	0.058498
8	0.122588	0.030240
16	0.061729	0.017037
32	0.031724	0.011754
48	0.022448	0.010830
96	0.017630	0.013609

Em 96 processos, a versao MPI sem BLAS continuou reduzindo o tempo, mas a versao MPI+OpenBLAS apresentou piora em relacao a 48 processos. Isso indica custo de comunicacao, desequilibrio ou saturacao para este tamanho de matriz. Os valores completos e a variabilidade estao nos CSVs multinode.

A primeira tentativa, 1032045.ada, solicitou os mesmos recursos, mas foi cancelada porque havia apenas 73 CPUs livres. Seus estados estao documentados junto com a submissao bem-sucedida em:

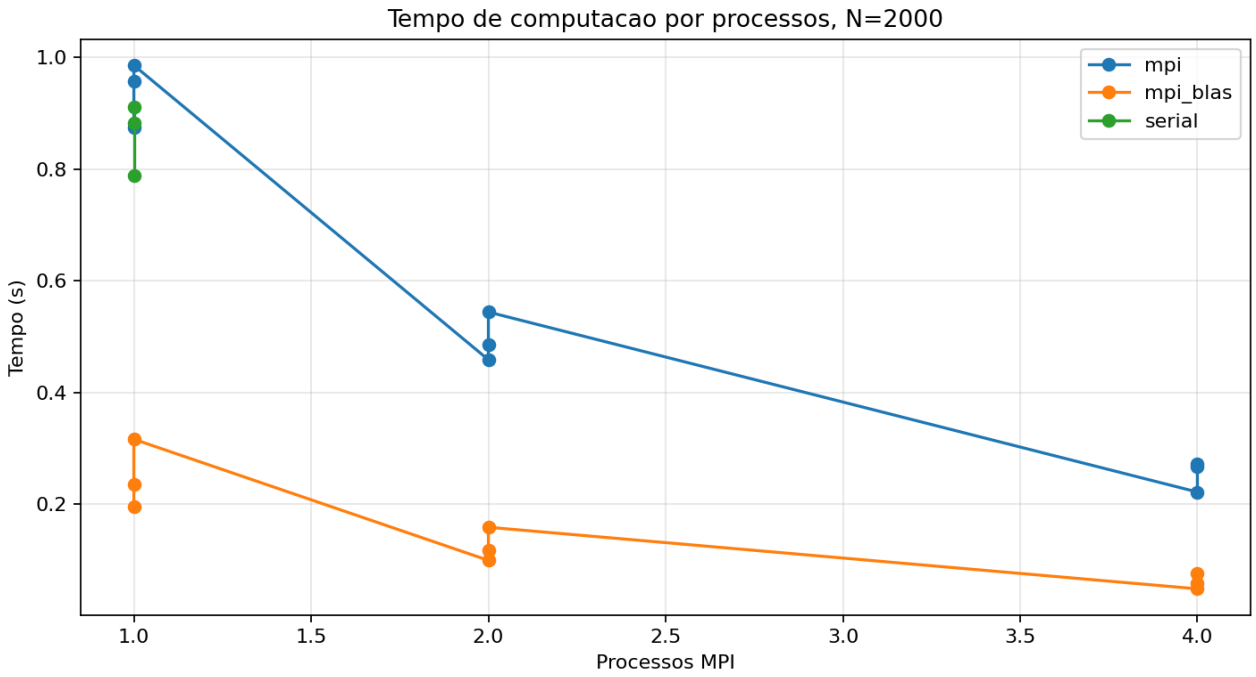
- results/environment/multinode-submission-1032047.ada.txt;
- results/environment/multinode-qstat-1032047.ada.txt;
- results/environment/multinode-qstat-latest-1032047.ada.txt;
- results/environment/multinode-attempts.txt.

O custo de comunicacao pode ser observado pela diferenca entre total_seconds e compute_seconds. No MPI+OpenBLAS multinode, essa diferenca foi aproximadamente 0.0041 s em p=1, 0.0221 s em p=8, 0.0437 s em p=48 e 0.0680 s em p=96 na primeira repeticao. O crescimento do overhead explica parte da piora em p=96 e inclui broadcast, distribuicao, sincronizacao e coleta.

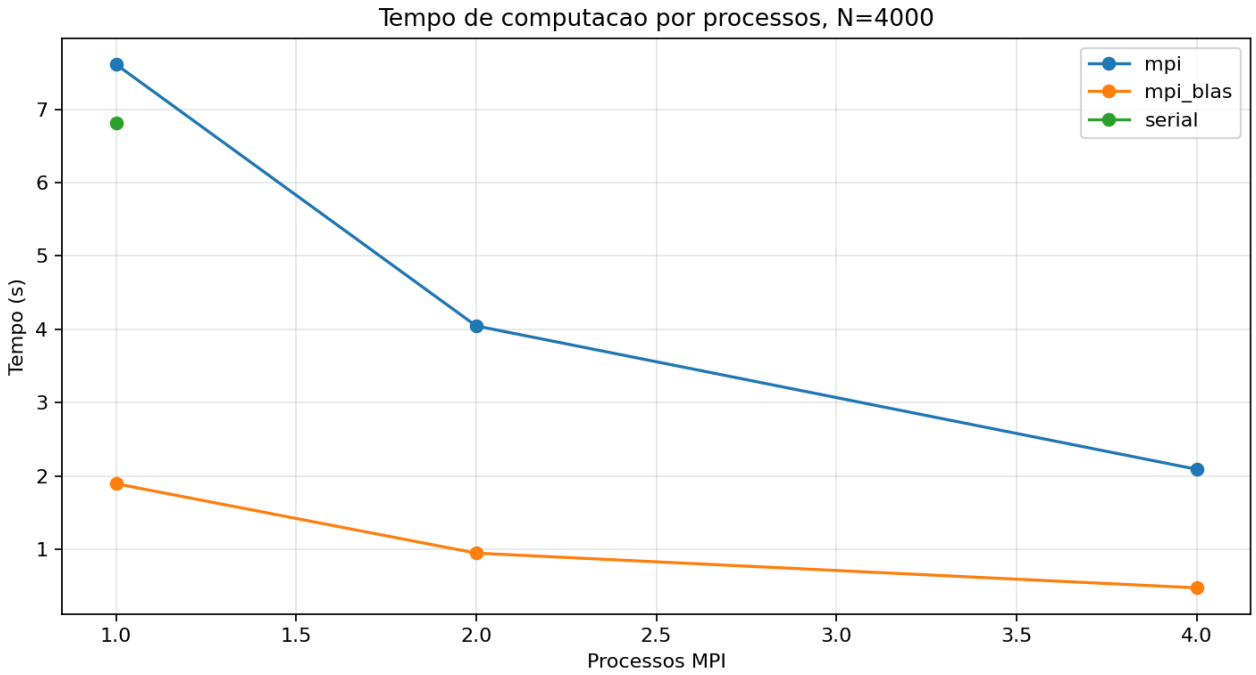
As tres matrizes float ocupam teoricamente cerca de 0.0447 GiB para N=2000, 0.1788 GiB para N=4000 e 0.4023 GiB para N=6000, sem contar buffers MPI e bibliotecas. Esses valores sao estimativas de armazenamento, nao medicoes de RSS; o inventario PBS e os logs preservam os recursos alocados.

Graficos de desempenho

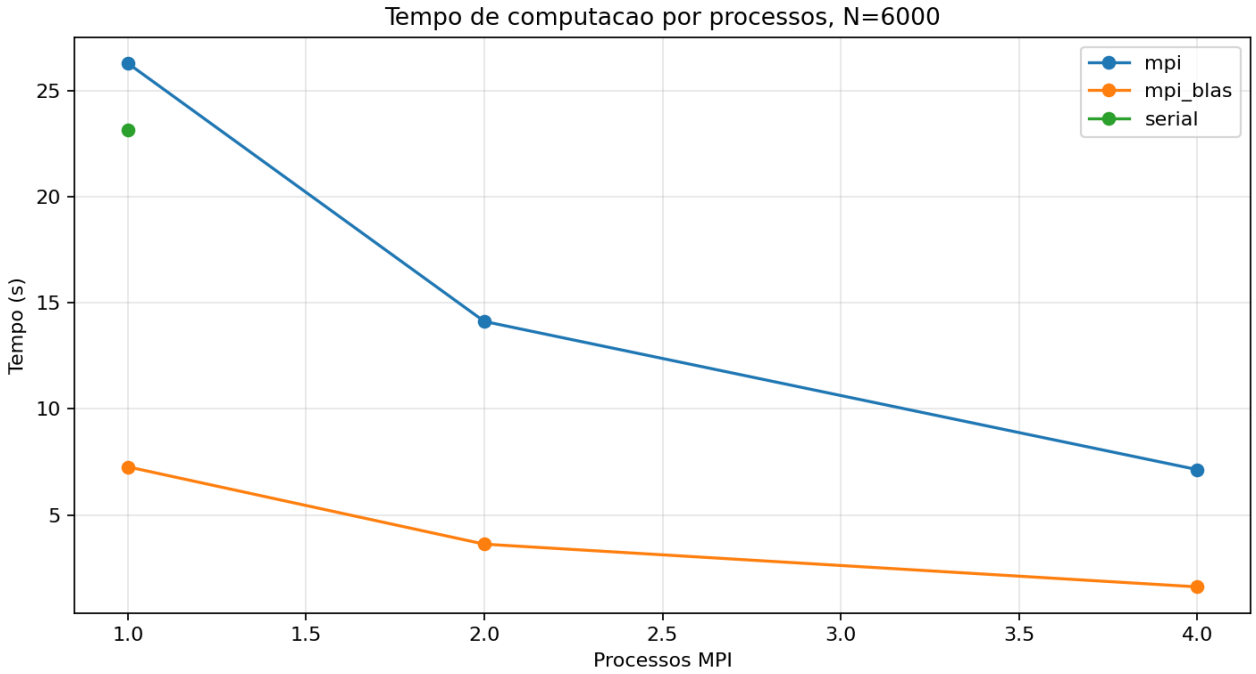
compute_time_n2000.png



compute_time_n4000.png



compute_time_n6000.png


Tempo de computacao por processos, N=6000
mpi
mpi_blas
serial
Tempo (s)
25
20
15
10
5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Processos MPI